

Teste Técnico — Transferências Financeiras

Vaga: Desenvolvedor .NET Pleno (Brasil Terrenos Holding)

Desenvolva uma API REST simples + interface frontend simples para realizar e agendar transferências entre contas. O objetivo é avaliar qualidade de código, organização, modelagem de domínio, regras de negócio e capacidade de lidar com situações comuns de um sistema financeiro.

1. Funcionalidade principal

- Permitir que uma pessoa transfira um valor para outra pessoa.
- Permitir agendar uma transferência para uma data futura.
- Consultar o status e os dados de uma transferência.
- Cancelar uma transferência enquanto ela estiver agendada.

2. Conta

- Uma pessoa possui uma conta.
- A conta possui saldo disponível.
- A conta possui um limite de cheque especial configurável.
- Uma conta bloqueada/inativa não pode realizar nem receber transferências.

3. Transferência imediata

- A conta de origem e a conta de destino devem estar com o status “ativo”.
- Origem e destino devem ser diferentes.
- O valor de transferência deve ser maior que zero.
- A conta de origem precisa possuir saldo + cheque especial suficiente.
- O débito da origem e o crédito do destino devem acontecer de forma atômica.

Exemplo: uma conta possui R\$ 500 de saldo e R\$ 1.000 de cheque especial. Ela pode realizar uma transferência de até R\$ 1.500.

4. Cheque especial

- A conta pode ficar negativa, desde que respeite seu limite de cheque especial.
- Exemplo: saldo de R\$ 500, transferência de R\$ 1.000 → novo saldo de -R\$ 500.
- Se o saldo estiver negativo e a conta receber uma transferência, o valor recebido deve primeiro cobrir o cheque especial utilizado.

Exemplo: saldo = -R\$ 800. Ao receber R\$ 1.000, R\$ 800 cobrem o cheque especial e R\$ 200 ficam disponíveis como saldo positivo.

5. Limite de transferência

- Cada pessoa possui um limite de transferência por hora.
- O limite pode ser diferente durante o dia e durante a noite.
- Também deve existir um limite de quantidade de tentativas por hora.
- Transferências rejeitadas devem contar como tentativa.

Exemplo de configuração:

Período	Valor máximo / hora	Tentativas / hora
Dia	R\$ 5.000	5
Noite	R\$ 1.000	3

6. Transferência agendada

- Deve ser possível informar conta de origem, destino, valor e data/hora.
- A data deve ser futura.
- O agendamento pode ser cancelado antes da execução.
- No momento da execução, saldo, cheque especial e limites devem ser verificados novamente.

7. Status

Uma transferência pode possuir os seguintes estados:

- Scheduled — agendada.
- Processing — em processamento.
- Completed — concluída.
- Failed — falhou.
- Cancelled — cancelada.

8. Regras técnicas

- Utilizar .NET 8 ou superior e C# para o backend, já para o frontend é de livre escolha do candidato dentre essas opções: ReactJS, Vue.JS ou AngularJS.
- Criar uma API REST.
- Utilizar banco relacional com Entity Framework Core.
- Disponibilizar Swagger/OpenAPI.
- A solução deve considerar o que acontece quando duas transferências são realizadas simultaneamente.

9. Arquitetura

Utilize uma arquitetura que facilite manutenção e evolução. O uso de DDD, SOLID e Design Patterns é desejável quando fizer sentido para o problema.

Não é necessário utilizar um Design Pattern específico. Você deve priorizar uma solução simples, clara e justificável.

10. Endpoints sugeridos

Método	Endpoint	Descrição
POST	/api/transfers	Realizar transferência
POST	/api/transfers/scheduled	Agendar transferência
POST	/api/transfers/{id}/cancel	Cancelar agendamento
GET	/api/transfers/{id}	Consultar transferência
GET	/api/accounts/{id}	Consultar conta

11. Cenários importantes

- Transferência com saldo suficiente.
- Transferência utilizando cheque especial.
- Transferência que ultrapassa o cheque especial.
- Recebimento enquanto a conta está utilizando cheque especial.
- Transferência acima do limite da hora.
- Excesso de tentativas dentro da hora.
- Transferência durante o período noturno.
- Transferência agendada sem saldo suficiente no momento da execução.
- Duas transferências simultâneas utilizando o mesmo saldo.

12. Entrega

- Código-fonte no GitHub.
- README com instruções para executar.
- Migrations.
- Breve explicação das decisões arquiteturais.

13. Diferenciais

- Docker.
- Testes de integração.
- Mensageria para processamento dos agendamentos.
- Background Worker ou Job para processamento dos agendamentos.
- Domain Events.
- Idempotência.

14. O que será avaliado

Critério	Avaliação
C# / .NET / Frontend	Qualidade e domínio da escrita.
Arquitetura	Separação de responsabilidades e organização.
DDD / SOLID	Modelagem e encapsulamento das regras.
Banco	Modelagem e consistência dos dados.
Regras de negócio	Implementação correta dos cenários.
Código	Legibilidade, simplicidade e facilidade de manutenção.

Observação: Não é necessário implementar todos os diferenciais. A solução principal deve priorizar clareza, corretude e qualidade.